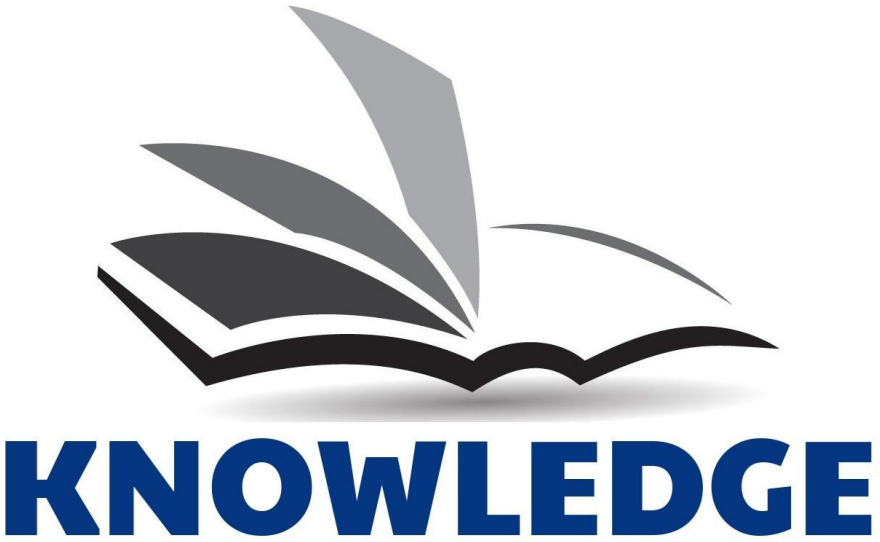
KNOWLEDGE LEARNING

Plateforme E-Learning



Projet réalisé dans le cadre du Titre Professionnel de Développeur Web et Web Mobile

Adeline CANON

Centre Européen de Formation – Mars 2026

SOMMAIRE

1. Contexte 3

2. Développer la partie Back End d'une application en intégrant les

Recommandations de sécurité 4

3. Résumé du projet 5

4. Cahier des charges 6

5. Charte graphique 10

6. Réalisation 10

7. Architecture de l'application 14

8. Spécificités techniques 14

9. Sécurité de l'application 15

10. Veille technologique 16

11. Conclusion 17

12. Annexes 18

1. Contexte

La société **Knowledge** est spécialisée dans la publication de supports de formation vendus en librairie. Afin d'élargir son offre et de s'adapter aux nouvelles pratiques d'apprentissage en ligne, l'entreprise souhaite développer une plateforme e-learning accessible depuis un site web.

Ce projet vise à permettre aux utilisateurs d'accéder à des formations directement en ligne, sans contrainte de lieu ou d'horaire. Les formations sont organisées en thèmes, cursus et leçons, permettant une progression structurée.

L'objectif du projet est donc de concevoir le **back-end d'une plateforme e-learning intégrant également des fonctionnalités e-commerce**, afin de permettre aux utilisateurs d'acheter et de suivre des formations en ligne.

2. Développer la partie Back End d'une application en intégrant les recommandations de sécurité

a. Création d'une base de données

J'ai structuré ma base de données en définissant les entités suivantes :

- ***User (Utilisateur) :*** représentant à la fois les clients de l'application et ses administrateurs.
- ***Theme (Thème) :*** les thèmes proposés à la vente.
- ***Cursus :*** les cursus mis en place dans les thèmes.
- ***Lesson (Leçon) :*** les leçons que contiennent les cursus.
- ***LessonValidated (Leçon validée) :*** afin de valider les leçons achetées et permettant l'obtention du certificat par la suite.
- ***Purchase (Achat) :*** le panier des clients.
- ***PurchaseItem (Article acheté) :*** Le détail d'un article acheté (cursus ou leçon).
- ***Certification :*** le certificat obtenu après la validation des cursus et/ou leçons.
- ***Contact :*** permettant aux utilisateurs de communiquer avec les administrateurs de la plateforme.

Voir Annexe : Schéma relationnel

b. Développer les composants d'accès aux données

L'accès aux données est assuré par **Doctrine ORM**, qui permet de manipuler les entités de l'application et d'interagir avec la base de données sans écrire directement de requêtes SQL. Doctrine facilite la gestion des relations entre les différentes entités et simplifie les opérations de lecture, création, modification et suppression des données.

c. Développer la partie Back End d'une application web

Le back-end de l'application a été développé avec le framework **Symfony**, qui permet d'organiser le code selon une architecture claire et maintenable.
Les **Controllers** gèrent les requêtes HTTP envoyées par les utilisateurs et orchestrent les différentes actions de l'application. Ils peuvent notamment récupérer des données via les **Repositories**, qui permettent d'accéder aux informations stockées dans la base de données grâce à **Doctrine ORM**.
Les **Entities** représentent les objets métiers de l'application et correspondent aux tables de la base de données. Les **Forms** permettent de gérer et valider les données saisies par les utilisateurs dans les différents formulaires du site.
Certaines fonctionnalités plus spécifiques sont regroupées dans des **Services**, afin de centraliser la logique métier et de garder les contrôleurs plus lisibles.
L'affichage des pages est assuré par les **templates Twig**, qui permettent de générer les pages HTML tout en séparant la logique applicative de la présentation.
Enfin, un système d'**authentification sécurisé basé sur des tokens** a été mis en place afin de protéger l'accès aux différentes fonctionnalités de l'application et de garantir la sécurité des utilisateurs.
Cette organisation permet de structurer l'application de manière claire, tout en facilitant son

évolution et sa maintenance.

3. Résumé du projet

Le projet **Knowledge Learning** consiste en la conception et le développement du back-end d'une plateforme e-learning intégrant des fonctionnalités e-commerce. L'objectif de cette application est de permettre aux utilisateurs d'accéder à des contenus pédagogiques structurés sous forme de thèmes, de cursus et de leçons, tout en offrant la possibilité d'acheter ces contenus en ligne.

La plateforme permet aux utilisateurs de créer un compte, de confirmer leur adresse e-mail puis de se connecter afin d'accéder aux différentes formations proposées. Chaque formation est organisée en plusieurs niveaux : un thème regroupe plusieurs cursus, eux-mêmes composés de plusieurs leçons. L'utilisateur peut acheter un cursus complet ou une leçon spécifique afin d'accéder aux contenus correspondants.

Une fois les leçons terminées, l'utilisateur peut valider sa progression. Lorsque toutes les leçons d'un cursus sont validées, celui-ci est automatiquement considéré comme complété et peut donner lieu à l'obtention d'une certification générée sous forme de document PDF.
L'application comprend également un système d'administration permettant de gérer les utilisateurs, les contenus pédagogiques, les achats ainsi que les messages envoyés via le formulaire de contact.

Le projet a été développé avec le framework Symfony **6.4**, en utilisant **PHP 8**, **Doctrine ORM** pour la gestion de la base de données MySQL, ainsi que **Twig** pour l'interface utilisateur. Des tests automatisés ont également été mis en place avec **PHPUnit** afin de garantir le bon fonctionnement des principales fonctionnalités de l'application.

4. Cahier des charges

a. Besoins et objectifs de l'application

La plateforme **Knowledge Learning** souhaite développer ses services en ligne afin de permettre aux utilisateurs de suivre des formations directement via une plateforme web.

Pour répondre à cet objectif, plusieurs besoins et fonctionnalités ont été identifiés :

- **Proposer une interface dynamique et intuitive**, permettant une navigation fluide afin que les utilisateurs puissent facilement trouver les formations et contenus recherchés.
- **Mettre à disposition un espace d'administration**, permettant aux administrateurs de gérer efficacement les contenus proposés en ligne (création, mise à jour, archivage des formations), mais également de consulter les achats, suivre la progression des utilisateurs et répondre aux demandes des clients.
- **Assurer la gestion des utilisateurs**, incluant l'inscription, la connexion sécurisée, la mise à jour des données personnelles, la consultation de l'historique des commandes ainsi que l'obtention de certificats après validation des leçons ou des cursus.
- **Mettre en place un système de paiement sécurisé via Stripe**, permettant l'achat de formations directement sur la plateforme.
- **Respecter la charte graphique et les contenus fournis par Knowledge Learning**, afin de garantir une cohérence visuelle et fonctionnelle avec l'identité de l'entreprise.

b. Structure du site

❖ Page d'accueil

L'application repose sur une **structure d'interface commune** à l'ensemble des utilisateurs, notamment avec un **header partagé** sur les différentes pages du site. Ce header contient le menu principal de navigation, mais son contenu est **personnalisé dynamiquement** en fonction du statut de l'utilisateur connecté.

Trois cas principaux sont distingués :
Utilisateur non connecté
le menu affiche les rubriques **Accueil**, **Thèmes**, **S'inscrire** et **Se connecter**.

Utilisateur connecté
le menu affiche les rubriques **Accueil**, **Thèmes**, **Panier**, **Dashboard**, **Contact** et **Déconnexion**.
Administrateur
le menu est adapté à son rôle et affiche principalement **Accueil**, **Dashboard** et **Déconnexion**.

Cette organisation permet de conserver une **base d'interface commune** tout en proposant une navigation **personnalisée selon le profil de l'utilisateur**, afin d'améliorer l'ergonomie, la clarté de l'interface et la sécurité d'accès aux différentes fonctionnalités.
Voir Annexe : Page d'accueil de la plateforme selon le statut de l'utilisateur (Non connecté, connecté et admin).

❖ Page Thème / Cursus / Leçons

Ces pages permettent aux utilisateurs de consulter les **formations proposées par Knowledge Learning**. Les contenus sont organisés en **thèmes**, eux-mêmes composés de **cursus**, qui contiennent plusieurs **leçons**. Les utilisateurs peuvent ainsi parcourir les différentes formations disponibles et sélectionner les contenus qu'ils souhaitent suivre.

❖ Page Panier

La page panier permet aux utilisateurs de **consulter les articles ajoutés**, de **modifier le contenu du panier** (ajout ou suppression d'éléments) et de **procéder au paiement**.Page Paiement

❖ Fonctionnalités E-Learning

L'utilisateur doit pouvoir :

- consulter les thèmes de formation mis à disposition en ligne
- accéder aux différents cursus proposés
- consulter les cours auxquels il a accès après achat
- valider les leçons ou les cursus suivis
- obtenir une certification après validation complète d'un cursus
- suivre sa progression dans les formations

❖ Fonctionnalités e-commerce

L'utilisateur doit être en mesure de :

- ajouter des éléments au panier
- supprimer des éléments du panier
- acheter des cursus ou des leçons
- accéder au contenu des cours achetés
- consulter l'historique de ses achats

❖ Fonctionnalités complémentaires

L'utilisateur peut également :

- obtenir des certificats après validation d'une leçon ou d'un cursus
- contacter les administrateurs du site via un formulaire de contact

Une **interface d'administration** permet également la gestion du contenu de la plateforme (utilisateurs, formations, achats, messages).

c. Base de données

L'application est une plateforme de formation en ligne permettant aux utilisateurs de consulter un catalogue de formations, d'acheter des cours et de suivre leur progression. Le catalogue est organisé de manière hiérarchique : **Theme → Cursus → Lesson**.
Les utilisateurs peuvent acheter des formations via un système de commandes (**Purchase** et **PurchaseItem**).
La progression est enregistrée grâce à **LessonValidated**, et des **certifications** peuvent être délivrées lorsque certaines formations sont complétées.

```
User
 ├── Purchases
 │      └── PurchaseItems
 │             ├── Lesson
 │             └── Cursus
 │
 ├── LessonValidated
 │      └── Lesson
 │
 └── Certifications
        ├── Lesson
        ├── Cursus
        └── Theme


Theme
 └── Cursus
        └── Lessons
```

d. Evolutions potentielles

Plusieurs améliorations pourraient être envisagées afin de faire évoluer la plateforme et d'enrichir l'expérience utilisateur.

❖ Personnalisation de l'expérience utilisateur

Un système de recommandation pourrait être mis en place afin de proposer automatiquement des cours ou des cursus pertinents en fonction de l'historique d'achat, des formations suivies ou des centres d'intérêt de l'utilisateur.

❖ Interaction et participation des utilisateurs

La plateforme pourrait intégrer un système permettant aux utilisateurs de **laisser des avis et des notes** sur les cours et les cursus. Cela permettrait d'aider les futurs apprenants à choisir leurs formations et d'améliorer la qualité globale des contenus.

❖ Outils d'analyse pour l'administrateur

L'ajout d'un module d'analyse permettrait à l'administrateur de suivre différents indicateurs tels que les **cours les plus consultés, les formations les plus achetées ou le taux de complétion des leçons**. Ces données pourraient être présentées sous forme de **tableaux de bord ou de rapports statistiques**.

❖ Amélioration du suivi pédagogique

La mise en place d'un **tableau de bord utilisateur** pourrait permettre de visualiser plus facilement la progression dans les formations, les leçons validées ainsi que les certifications obtenues.

Ces évolutions permettraient d'améliorer l'expérience des utilisateurs, de renforcer l'engagement des apprenants et d'offrir à l'administrateur de meilleurs outils de pilotage de la plateforme.

5. Charte graphique et Logo

Dans le cadre du projet, **Knowledge Learning a fourni une palette de couleurs à utiliser afin de respecter l'identité visuelle de la plateforme**. Ces couleurs ont été définies sous forme de variables CSS afin de faciliter leur utilisation et leur maintenance dans l'ensemble du site.
La couleur **bleue principale (#0074c7)** est utilisée pour les éléments interactifs tels que les boutons et les liens, tandis que sa version **plus foncée (#00497c)** sert aux états actifs ou au survol. Le **bleu très clair (#f1f8fc)** est utilisé pour les arrière-plans de certaines interfaces, notamment l'espace administrateur. Les couleurs **gris foncé (#384050)** et **bleu transparent** sont utilisées respectivement pour le texte, les bordures et les ombres afin d'assurer une bonne lisibilité et une interface visuellement cohérente.

#f1f8fc	#0074c7	#00497c	#384050	#cd2c2e	#82b864

6. Réalisation

a. Fonctionnalité principale : achat d'une leçon

L'utilisateur peut acheter une leçon depuis le catalogue de la plateforme. Lorsqu'une leçon est ajoutée au panier, une commande (`Purchase`) est créée avec le statut **cart**, et l'élément acheté est enregistré dans l'entité `PurchaseItem`.
Lors de la validation du paiement, l'application vérifie que le panier contient des éléments, calcule le montant total de la commande puis met à jour son statut en **paid** afin de confirmer l'achat. L'utilisateur est ensuite redirigé vers une page de confirmation affichant le numéro de commande.

❖ Processus d'achat

- L'utilisateur ajoute une leçon à son panier
- Une commande (`Purchase`) avec le statut **cart** est créée
- La leçon est enregistrée dans `PurchaseItem`
- L'utilisateur valide le paiement
- Le total de la commande est calculé
- Le statut de la commande passe à **paid**
- Une page de confirmation est affichée

❖ Extrait de code principal

```
$purchase->calculateTotal();
$purchase->markPaid();

$this->em->flush();
```

b. Paiement et validation de la commande

Une fois le panier validé, l'application calcule le montant total de la commande. Le statut de la commande passe alors de **"cart"** à **"paid"**, ce qui confirme le paiement. Les modifications sont ensuite enregistrées en base de données et l'utilisateur est redirigé vers une page de confirmation affichant le numéro de commande.

c. Accès aux cours achetés

Une fois la commande payée, l'utilisateur peut accéder aux leçons qu'il a achetées.
Le système vérifie :

- l'utilisateur est authentifié ;
- la leçon demandée appartient à une commande dont le statut est **paid**.

Cette vérification permet de garantir que **seuls les utilisateurs ayant acheté la formation peuvent accéder au contenu pédagogique**.

❖ **Extrait de code principal**

```
$purchase->calculateTotal();
$purchase->markPaid();

$this->em->flush();
```

d. Validation d'une leçon

Lorsqu'une leçon est terminée, l'utilisateur peut la valider.
Cette action crée un enregistrement dans l'entité **LessonValidated**, qui contient :

- l'utilisateur concerné
- la leçon validée
- la date de validation

Ce mécanisme permet de **suivre la progression de l'utilisateur au sein de la formation**.

e. Obtention d'un certificat

Lorsque certaines conditions sont remplies (par exemple lorsque toutes les leçons d'un cursus sont validées), un **certificat** peut être attribuée à l'utilisateur.
L'entité **Certification** enregistre les informations suivantes :

- l'utilisateur
- la formation concernée
- la date d'obtention
- un code unique de certificat

Ce certificat permet d'attester que l'utilisateur a terminé la formation avec succès.
Voir Annexe : Les Controllers et les méthodes complètes implémentant ces fonctionnalités

f. Gestion des rôles et sécurité des accès

Pour assurer la sécurité de l'application et contrôler les différents niveaux d'accès, un système de rôles a été mis en place à l'aide du composant **Symfony Security**. Trois types de profils utilisateurs sont distingués dans l'application :

- **User (client)** : utilisateur standard de la plateforme. Il peut accéder aux thèmes, cursus, leçons, au panier ainsi qu'à son espace personnel.
- **User Admin (administrateur)** : utilisateur disposant de droits supplémentaires lui permettant de gérer le contenu de la plateforme (thèmes, cursus, leçons et gestion des utilisateurs).
- **User Archived** : utilisateur dont le compte a été archivé par un administrateur. Dans ce cas, l'accès à l'application est bloqué.

Cette gestion des rôles repose sur plusieurs éléments techniques du projet.

❖ Stockage des rôles dans l'entité User

Les rôles sont stockés dans l'entité `User`. Chaque utilisateur possède un tableau de rôles permettant de définir ses permissions dans l'application.
La méthode `getRoles()` garantit que chaque utilisateur possède au minimum le rôle `ROLE_USER`, même si aucun rôle spécifique n'est stocké en base de données.

```php
public function getRoles(): array
{
    $roles = $this->roles;
    $roles[] = 'ROLE_USER';

    return array_values(array_unique($roles));
}
```

Ainsi :

- un **client** possède le rôle `ROLE_USER`
- un **administrateur** possède le rôle `ROLE_ADMIN`
- tous les utilisateurs disposent au minimum du rôle `ROLE_USER`

Cette approche permet de simplifier la gestion des permissions dans le reste de l'application.

❖ Vérification des rôles dans les controllers

En complément de la configuration globale, les contrôleurs utilisent également des vérifications de rôles afin d'adapter le comportement de l'application selon le type d'utilisateur.
Par exemple, dans le contrôleur des leçons :

```php
#[Route('/lesson/{id}', name: 'lesson_show', methods: ['GET'])]
public function show(int $id, LessonRepository $lessonRepository): Response
{
    if ($this->isGranted('ROLE_ADMIN')) {
        return $this->redirectToRoute('admin_dashboard');
    }
```

Dans ce cas, si un administrateur tente d'accéder à une page destinée aux clients, il est automatiquement redirigé vers le tableau de bord d'administration.

Les contrôleurs vérifient également si l'utilisateur est connecté :

```
$user = $this->getUser();
if (!$user instanceof User) {
    return $this->redirectToRoute('app_login');
}
```

Cette vérification empêche un visiteur non authentifié d'accéder aux contenus réservés aux utilisateurs.

❖ Gestion des comptes archivés

Un mécanisme supplémentaire de sécurité a été mis en place à l'aide d'un **UserChecker personnalisé**. Ce composant permet de vérifier l'état du compte au moment de l'authentification.
Si un compte est archivé ou non vérifié, la connexion est automatiquement refusée.

```
if ($user->isArchived()) {
    throw new CustomUserMessageAccountStatusException(
        'Votre compte est archivé. Contactez un administrateur.'
    );
}
```

De la même manière, un utilisateur dont l'adresse email n'a pas été vérifiée ne pourra pas se connecter.
Ce système permet à l'administrateur de **désactiver un compte sans le supprimer**, ce qui facilite la gestion des utilisateurs.

❖ Utilisation des datas fixtures

Des **data fixtures** ont été mises en place afin de générer automatiquement des données de test dans la base de données. Elles permettent de préparer rapidement un environnement cohérent pour le développement et l'exécution des tests automatisés.
Plusieurs fixtures sont utilisées dans le projet :

- **ThemeFixtures** : création de thèmes, cursus et leçons.
- **TestUserFixtures** : génération d'utilisateurs de test avec différents rôles.
- **PurchaseFixtures** : création d'achats fictifs pour tester l'accès aux contenus.
- **ContactFixtures** : génération de messages de contact.

Ces fixtures sont principalement utilisées dans les **tests automatisés (PHPUnit)** afin de charger des données fiables avant l'exécution des scénarios de test et de garantir la reproductibilité des résultats.

7. Architecture de l'application

L'application a été développée en suivant l'architecture **MVC (Model View Controller)** utilisée par le framework Symfony.
Le **Model** correspond aux entités et aux repositories qui gèrent l'accès aux données via Doctrine ORM.
Les **Controllers** traitent les requêtes HTTP des utilisateurs et orchestrent la logique applicative.
Les **Views** sont générées à l'aide des templates Twig, responsables de l'affichage des pages.
Cette architecture permet de séparer les responsabilités de l'application, ce qui améliore la lisibilité du code, facilite la maintenance et permet de faire évoluer le projet plus facilement.

8. Spécificités Techniques

Le projet a été développé avec **Symfony 6.4** et **PHP 8**, permettant une architecture structurée et maintenable. La gestion des données est assurée par **Doctrine ORM** avec une base de données **MySQL**. L'interface utilisateur est générée à l'aide du moteur de templates **Twig**. Enfin, des **tests automatisés avec PHPUnit** ont été mis en place afin de vérifier le bon fonctionnement des principales fonctionnalités de l'application.

a. Gestion des routes

Les routes de l'application sont définies directement dans les contrôleurs à l'aide des attributs Symfony (`#[Route]`). Cette approche permet d'associer clairement chaque URL à une action du contrôleur. Elle facilite également la maintenance et la compréhension de la structure de l'application.

b. Gestion des données

La gestion des données est assurée par **Doctrine ORM**, qui permet de manipuler les informations de la base MySQL sous forme d'objets. Les principales entités du projet sont notamment `User`, `Theme`, `Cursus`, `Lesson`, `Purchase` et `Certification`.

c. Gestion du panier

Un système de panier permet aux utilisateurs d'ajouter des leçons ou des cursus avant de procéder à l'achat. Les achats sont enregistrés dans les entités `Purchase` et `PurchaseItem`, permettant de gérer les éléments du panier, le paiement et l'accès aux contenus achetés.

d. Interface utilisateur

L'interface utilisateur est générée grâce au moteur de templates **Twig**, permettant de produire des pages HTML dynamiques. Les templates sont organisés de manière à réutiliser certains éléments communs, comme le menu ou les composants d'interface.

e. Tests

Des tests automatisés ont été réalisés avec **PHPUnit** afin de vérifier le bon fonctionnement des principales fonctionnalités. Les tests couvrent notamment la sécurité, l'accès aux contenus, l'authentification et la gestion du panier.

f. Gestion du projet

Le code source du projet **Knowledge Learning** est disponible sur la plateforme GitHub afin de faciliter l'accès au projet, sa consultation ainsi que son installation.
Le dépôt contient l'ensemble du code source de l'application ainsi que les fichiers nécessaires à sa configuration et à son exécution.

Lien vers le Repository GitHub :
https://github.com/Ad15C/Knowledge_learning.git

Un fichier **README.md** est présent à la racine du projet. Ce document constitue la documentation principale du projet et fournit toutes les instructions nécessaires pour installer et lancer l'application dans un environnement de développement.
Le README décrit notamment les étapes suivantes :

- les prérequis nécessaires (PHP, Composer, MySQL, Symfony CLI)
- l'installation des dépendances du projet
- la configuration de l'environnement
- la création et la configuration de la base de données
- le chargement des données de test
- le lancement de l'application
- l'exécution des tests automatisés

La documentation du projet est également disponible dans le dossier **docs/** du Repository. Elle contient des informations complémentaires concernant l'architecture de l'application, l'organisation du code, les workflows métier ainsi que certaines décisions techniques prises lors du développement.

g. Déploiement

L'application a été déployée sur un serveur web permettant son accès en ligne. Le déploiement inclut l'installation des dépendances Symfony, la configuration de la base de données et la mise en place de l'environnement de production.

9. Sécurité de l'application

Plusieurs mécanismes de sécurité ont été mis en place dans l'application. Symfony assure notamment la protection contre certaines vulnérabilités courantes telles que les attaques **CSRF**, les injections **SQL** (grâce à Doctrine ORM) et la gestion sécurisée de l'authentification des utilisateurs.
L'authentification repose sur un système de **sessions sécurisées** associé à des **tokens CSRF** pour protéger les formulaires.
Des contrôles d'accès basés sur les rôles (`ROLE_USER`, `ROLE_ADMIN`) permettent également de restreindre l'accès aux différentes fonctionnalités de l'application.

10. Veille technologique

Une veille technologique a été réalisée tout au long du projet afin de suivre les bonnes pratiques de développement avec Symfony et PHP. Elle s'est appuyée sur la documentation officielle Symfony, ainsi que sur différentes ressources en ligne et forums de développement.

11.Conclusion

La réalisation du projet **Knowledge Learning** a permis de mettre en pratique plusieurs compétences liées au développement d'applications web avec le Framework Symfony.

Ce projet a notamment permis de travailler sur la conception d'une **base de données relationnelle**, l'architecture d'une application web structurée, la mise en place d'un système **e-learning couplé à des fonctionnalités e-commerce**, ainsi que la gestion des utilisateurs et des droits d'accès.

La mise en place de **tests automatisés avec PHPUnit** a également constitué une partie importante du projet. Leur implémentation a permis de garantir la fiabilité de l'application et de vérifier le bon fonctionnement des différentes fonctionnalités. La maintenance et la mise à jour régulière de ces tests ont cependant nécessité une attention particulière afin de conserver une cohérence globale entre les différentes parties de l'application.

Plusieurs difficultés ont été rencontrées au cours du développement. Il a notamment été nécessaire de gérer différents **niveaux d'accès utilisateurs**, tels que les visiteurs non connectés, les utilisateurs connectés n'ayant pas effectué d'achat, les utilisateurs ayant acheté des contenus, ainsi que les administrateurs disposant de droits étendus sur la plateforme. La gestion de ces différents rôles et des droits associés a demandé une réflexion approfondie afin de garantir la sécurité et la cohérence de l'application.

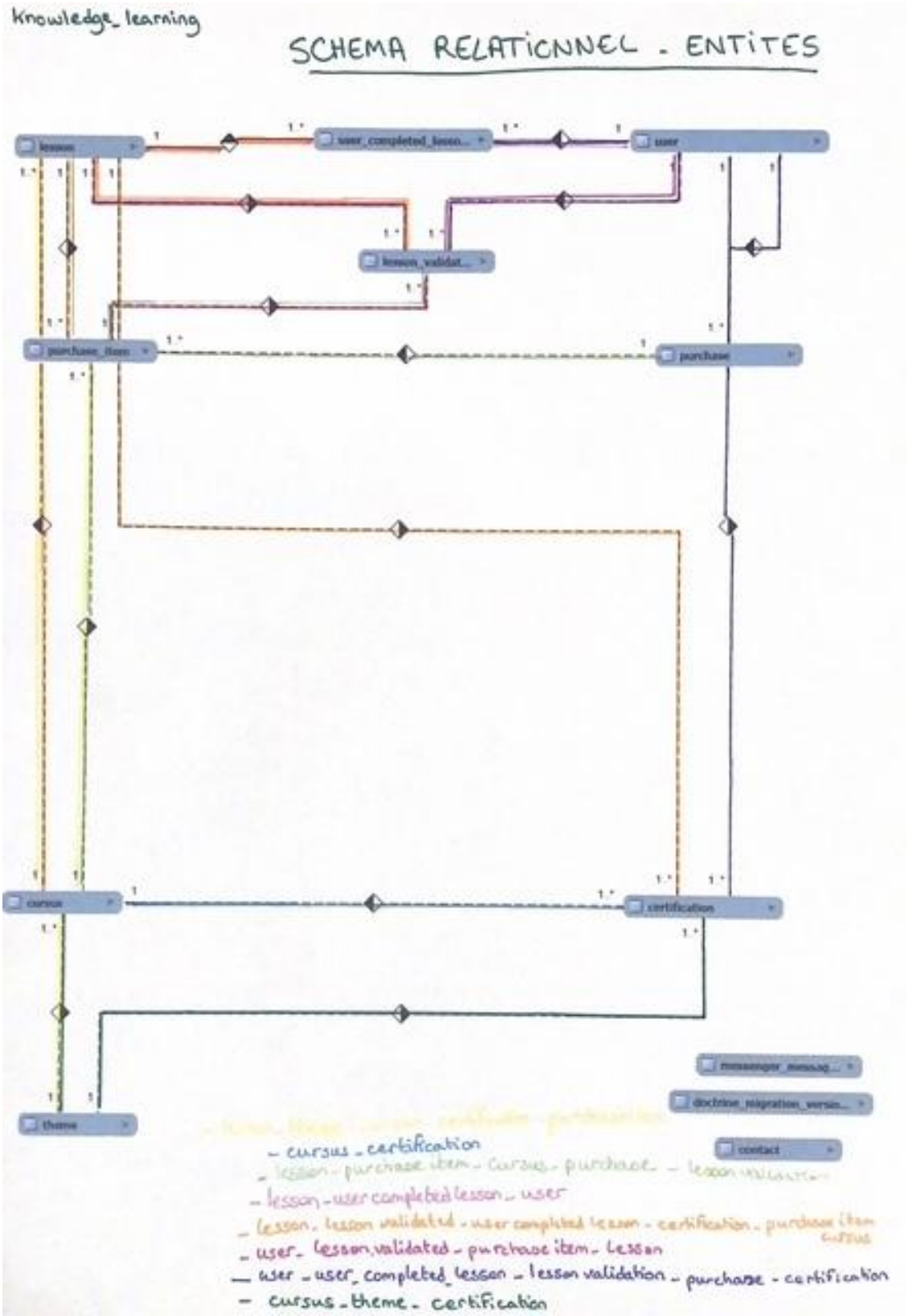
D'autres aspects du projet ont également représenté des défis techniques, notamment la gestion simultanée des fonctionnalités côté utilisateur et côté administration, la mise en place d'une interface adaptable à différents formats d'écran ainsi que l'implémentation des différents tests nécessaires au bon fonctionnement de l'application.
Certaines améliorations pourraient être envisagées pour faire évoluer le projet. Il serait par exemple possible d'intégrer un **système de paiement réel**, le projet utilisant actuellement une simulation de paiement basée sur Stripe. Une amélioration de la structure du code et de l'organisation du projet pourrait également être étudiée afin d'optimiser la maintenabilité de l'application.

Au cours du développement du projet, certaines contraintes techniques liées à l'environnement de développement ont également été rencontrées. L'installation complète de l'environnement nécessaire au projet sous Windows s'est révélée plus complexe que prévu. Afin de garantir le bon fonctionnement des outils nécessaires au développement, une configuration simulant un environnement Linux a été utilisée.
Cette solution a permis de contourner certaines incompatibilités rencontrées lors de l'installation des dépendances et de poursuivre le développement du projet dans des conditions stables.

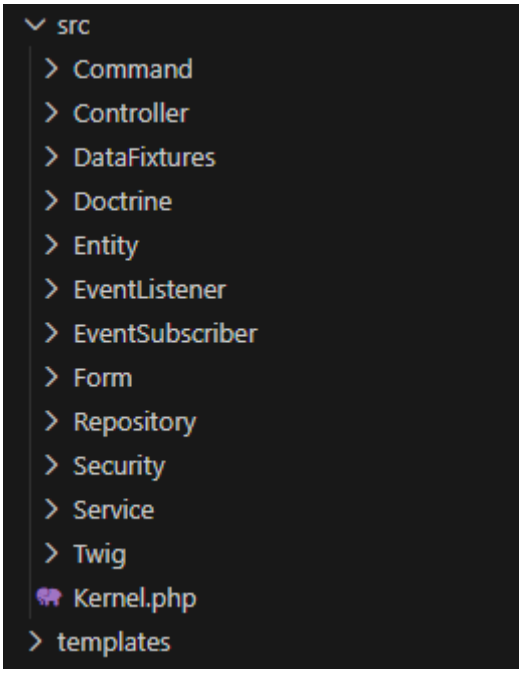
Cette expérience a également permis de mieux comprendre l'importance de la **gestion de l'environnement de développement** et des différences pouvant exister entre les systèmes d'exploitation lors de la mise en place d'un projet web.

12. Annexes

a. Schéma relationnel de la base de données

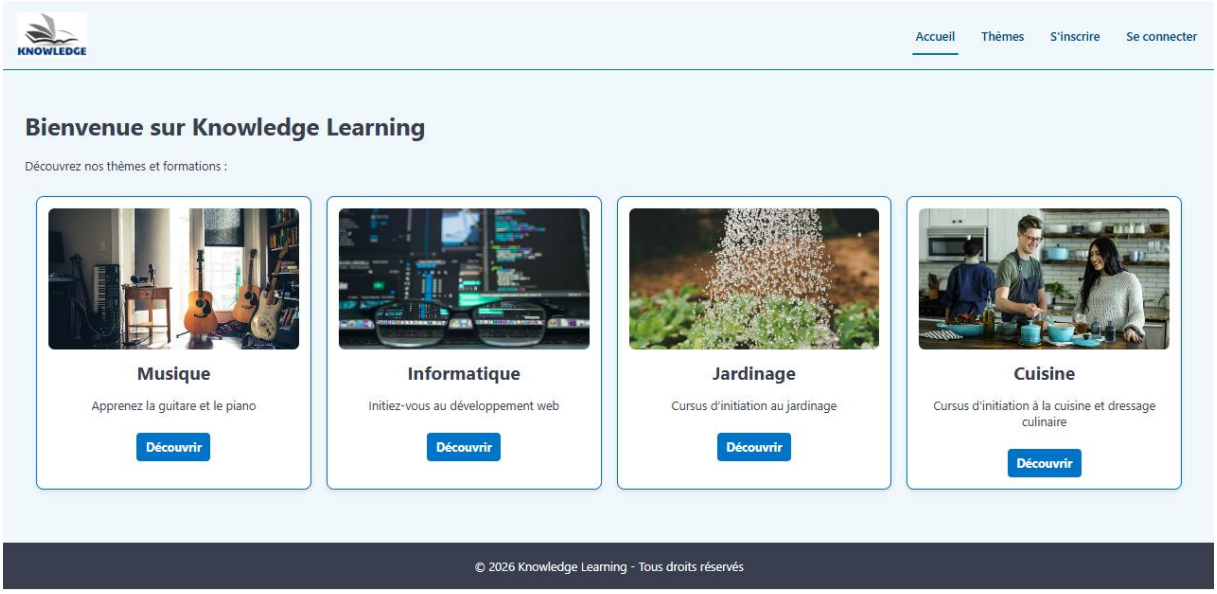


b. Structure du dossier *src* de l'application

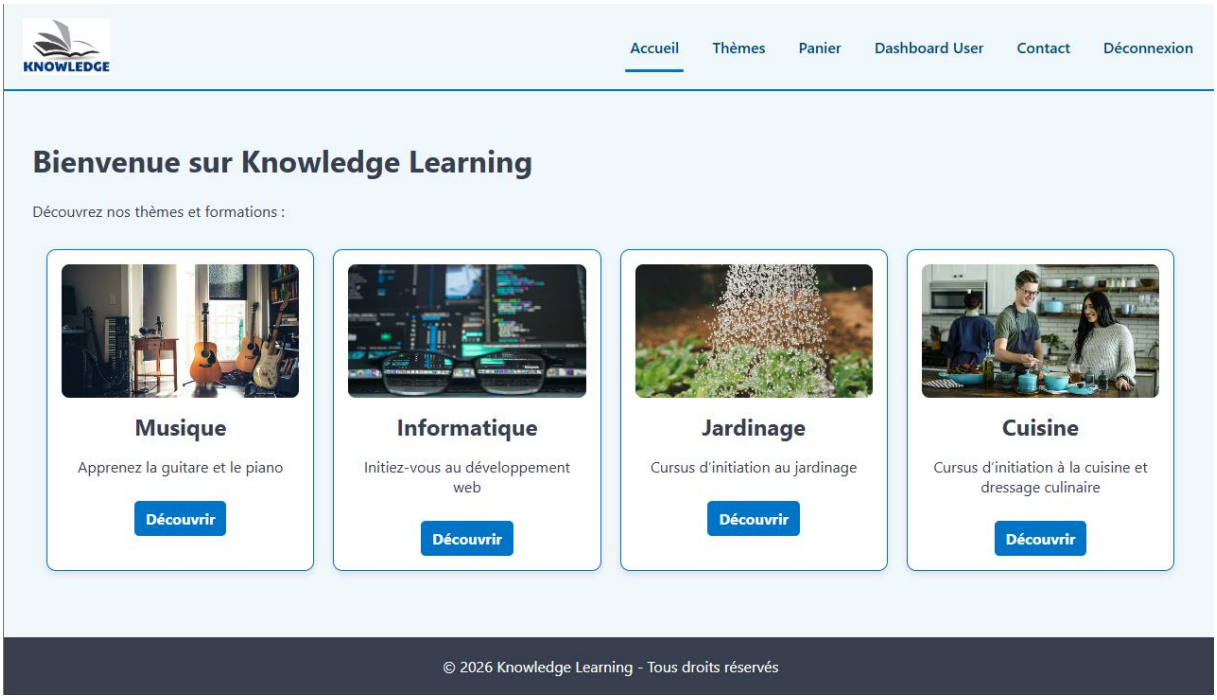


c. Interface de la page d'accueil selon le statut de l'utilisateur

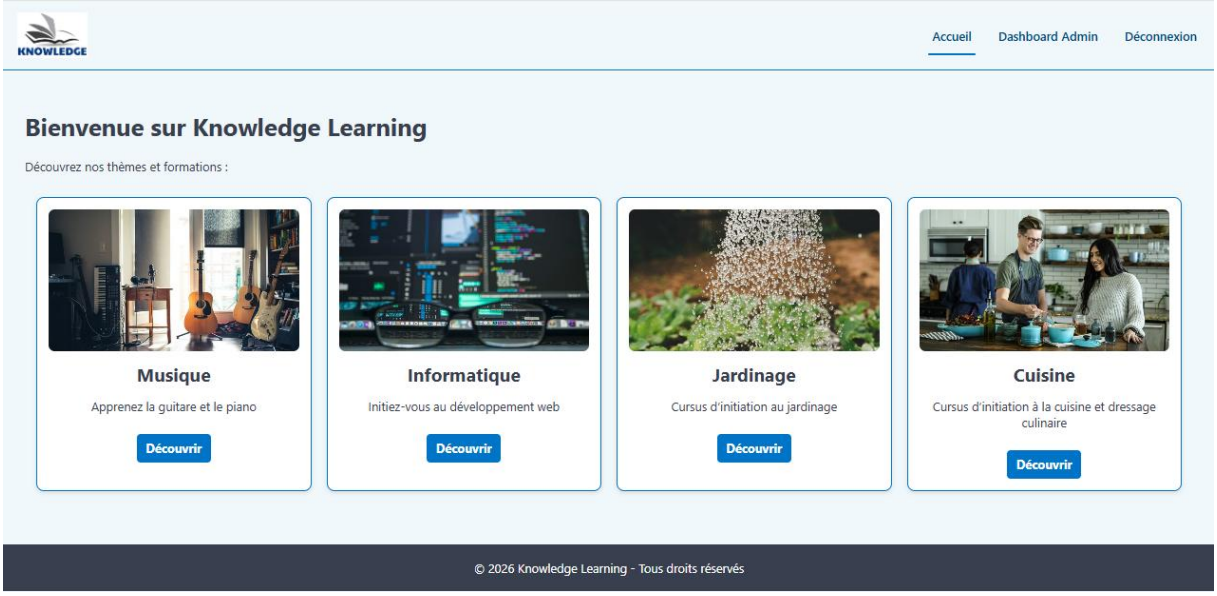
Utilisateur non connecté



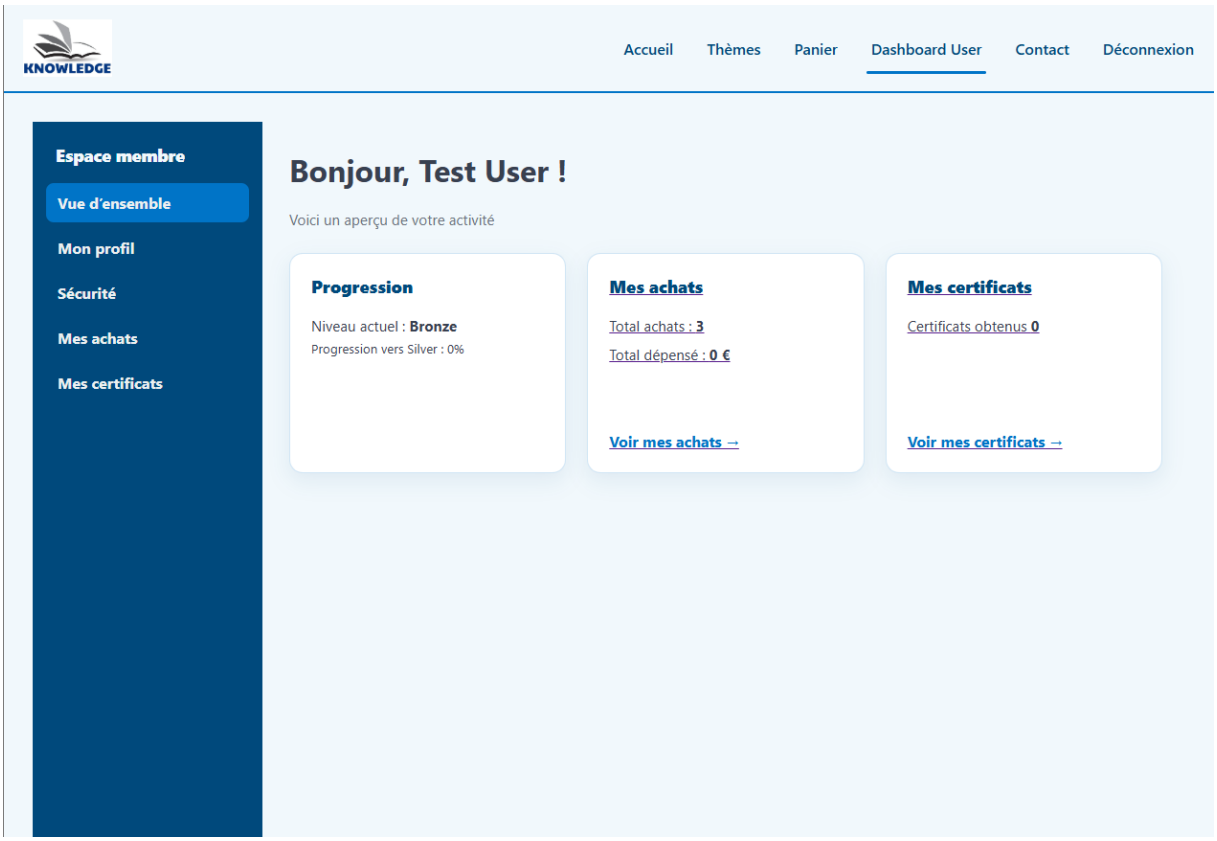
Utilisateur connecté



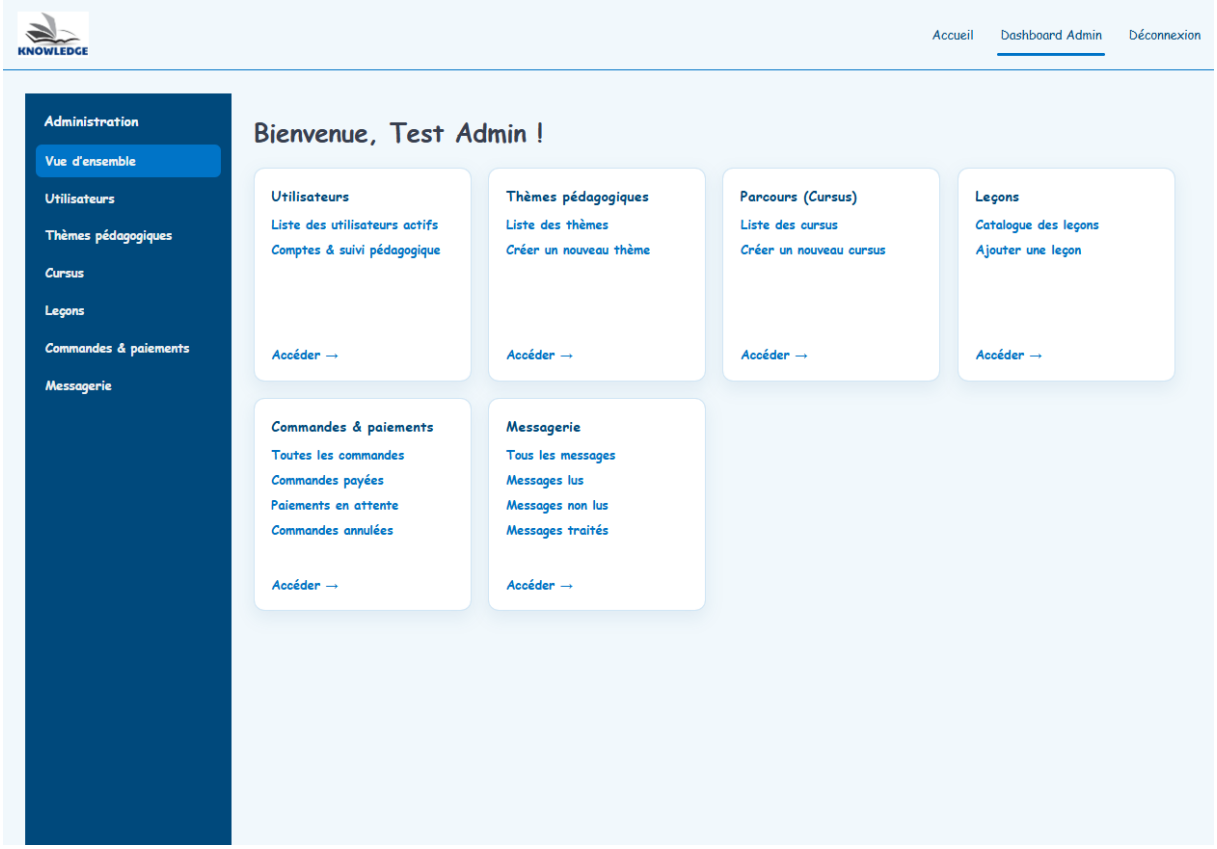
Admin connecté



d. Interface de l'espace membre

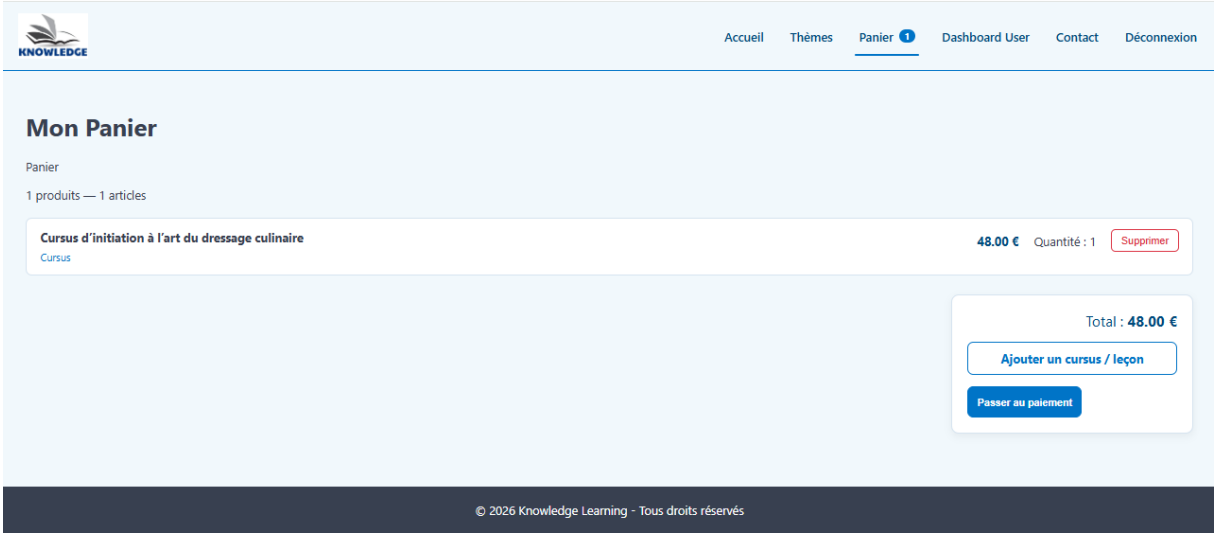


e. Interface de l'espace admin

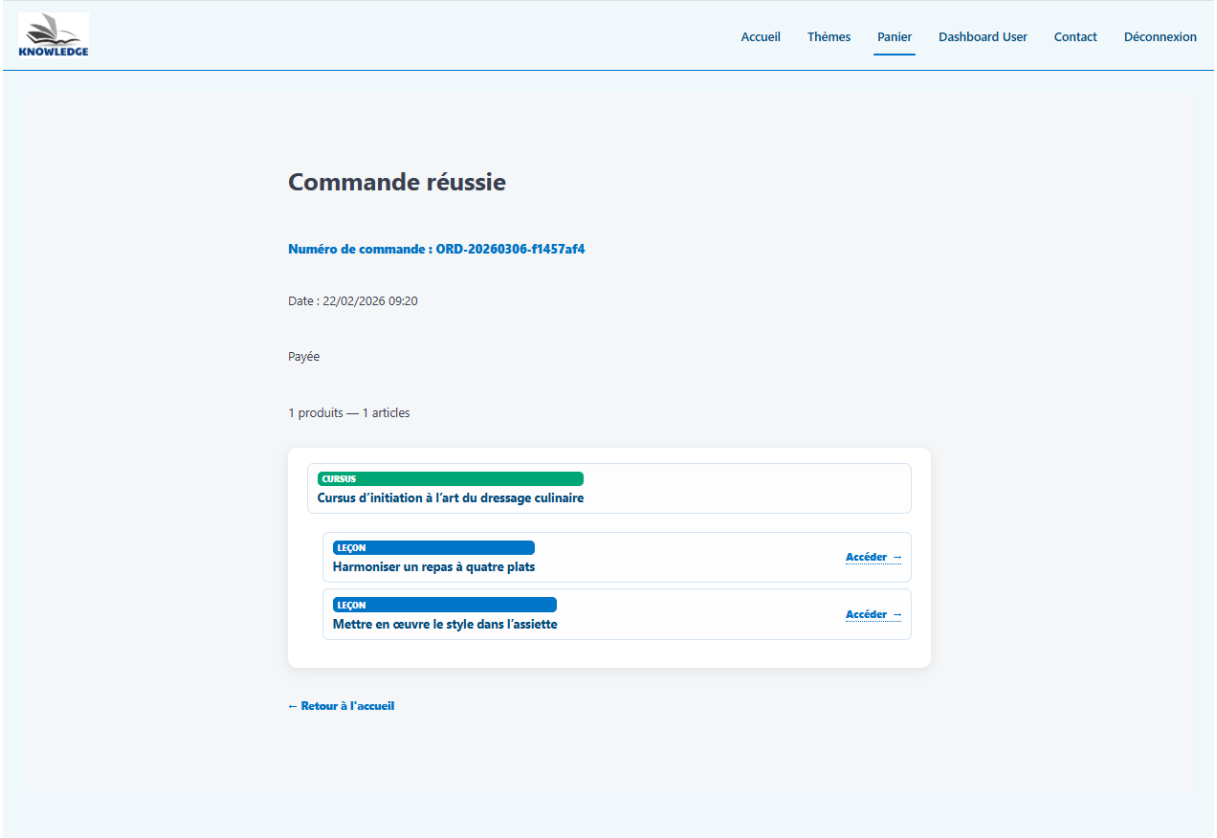


f. Interface du panier et validation de paiement

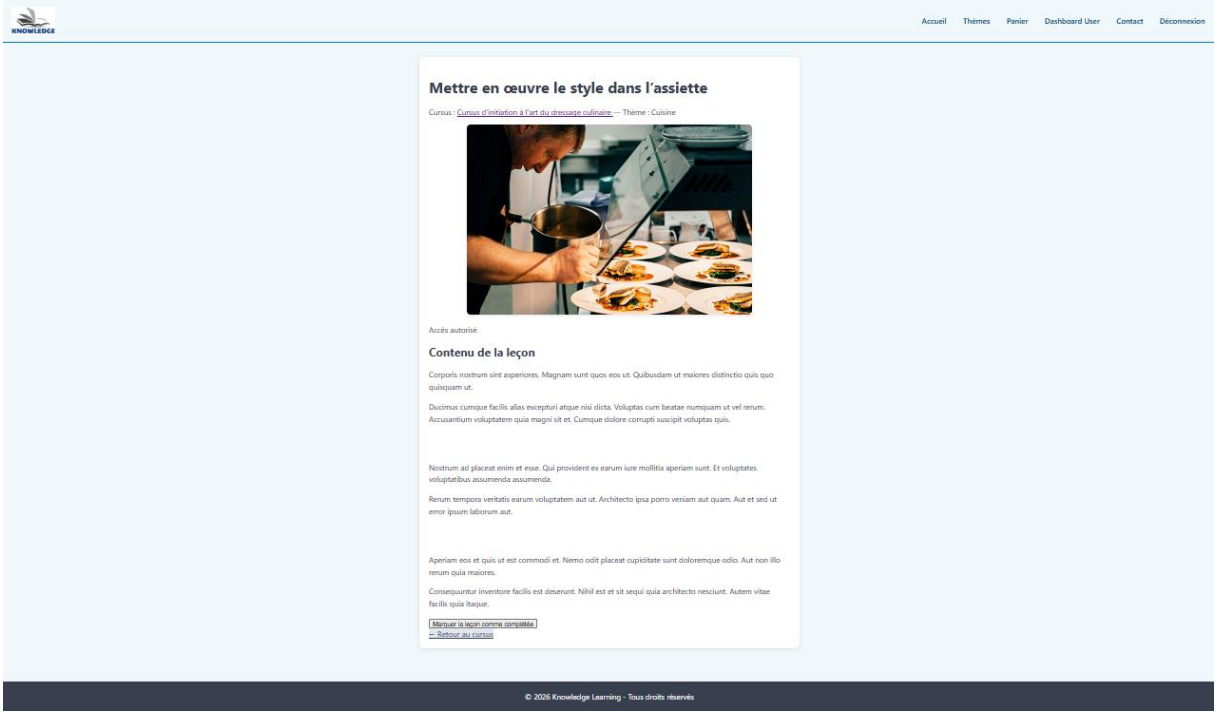
Panier de l'utilisateur



Page de validation de paiement

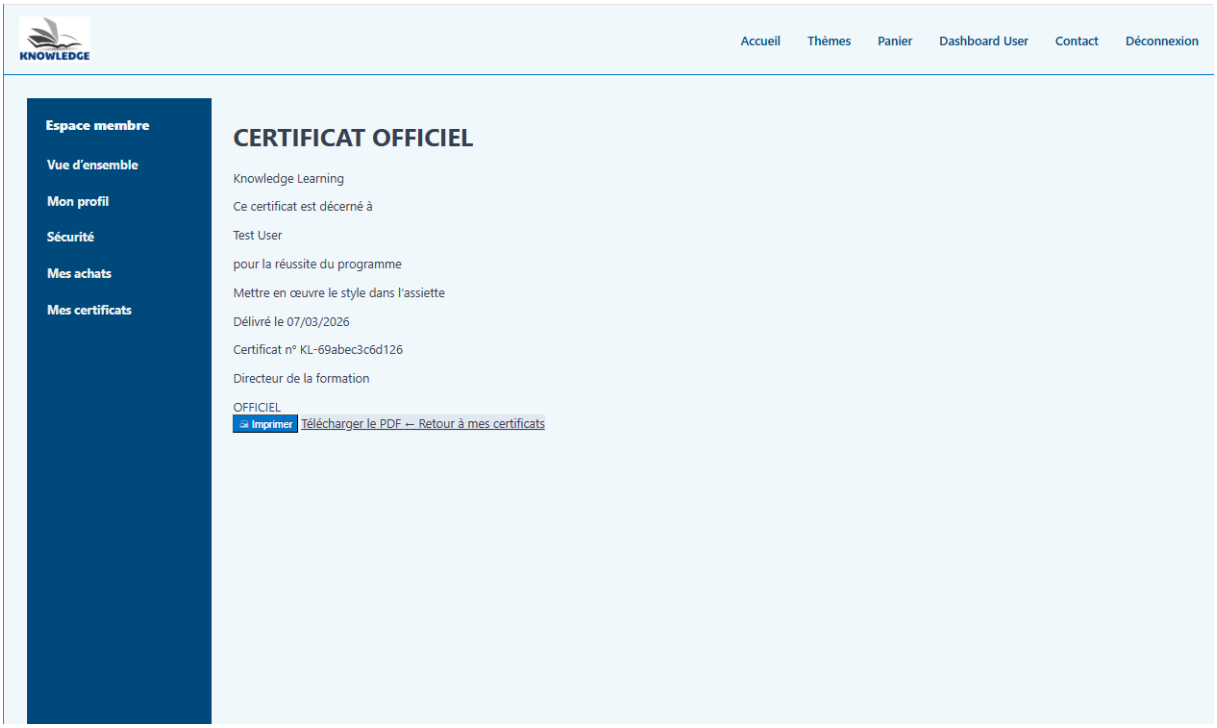


g. Interface de la page de leçon selon le statut de l'utilisateur

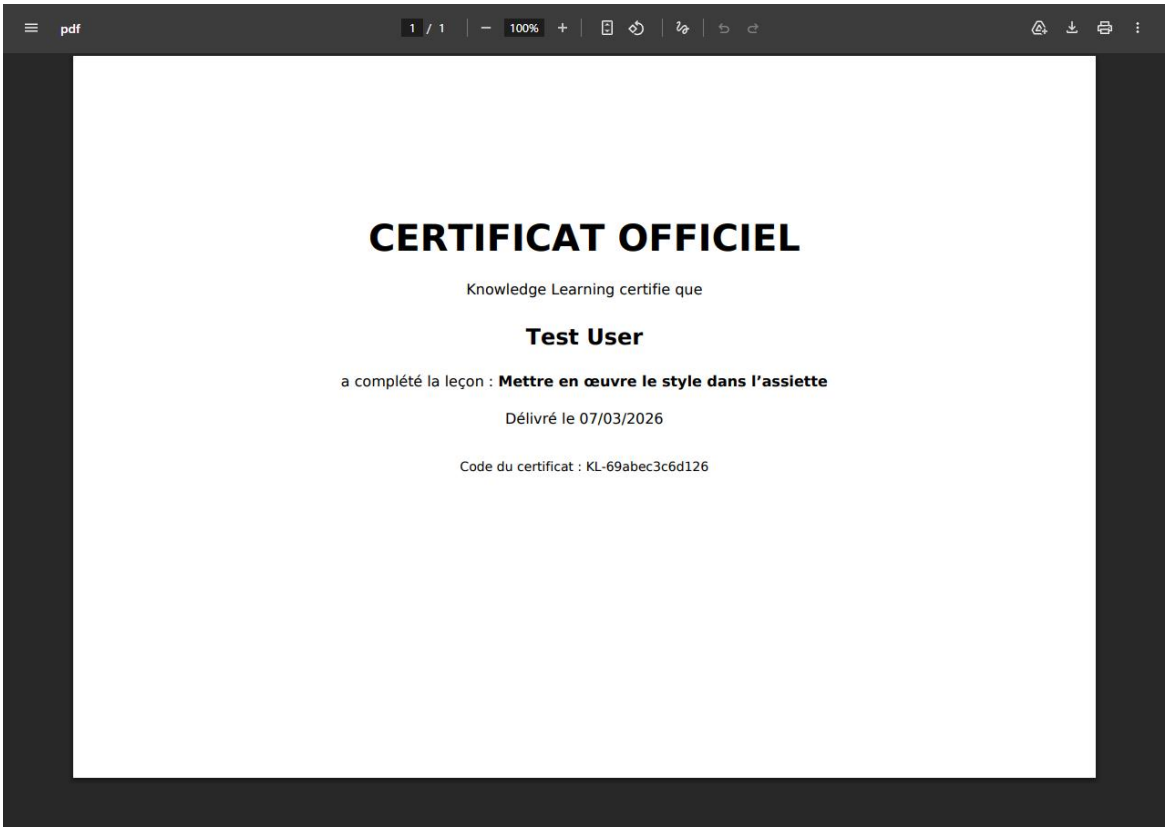


h. Interface de la page de certification et sa version PDF

Accès au certificat après la validation de la leçon



Certificat PDF généré



i. Extraits de code des controllers principaux

PurchaseController montrant la route pour voir le panier

```php
#[Route('/cart', name: 'cart_show', methods: ['GET'])]
public function show(): Response
{
    if ($response = $this->redirectAdminToDashboard()) {
        return $response;
    }

    $user = $this->getConnectedUser();
    if (!$user) {
        return $this->redirectToRoute('app_login');
    }

    $purchase = $this->purchaseRepo->findOneBy([
        'user' => $user,
        'status' => Purchase::STATUS_CART,
    ]);

    return $this->render('cart/show.html.twig', [
        'purchase' => $purchase,
    ]);
}
```

Route pour le paiement du panier

```
#[Route('/cart/pay', name: 'cart_pay', methods: ['POST'])]
public function pay(Request $request): Response
{
    if ($response = $this->redirectAdminToDashboard()) {
        return $response;
    }

    if (!$this->isCsrfTokenValid('cart_pay', (string) $request->request->get('_token'))) {
        throw $this->createAccessDeniedException('Invalid CSRF token.');
    }

    $user = $this->getConnectedUser();
    if (!$user) {
        return $this->redirectToRoute('app_login');
    }

    $purchase = $this->purchaseRepo->findOneBy([
        'user' => $user,
        'status' => Purchase::STATUS_CART,
    ]);

    if (!$purchase || $purchase->getItems()->isEmpty()) {
        $this->addFlash('error', 'Panier vide.');
        return $this->redirectToRoute('cart_show');
    }

    $purchase->calculateTotal();
    $purchase->markPaid();

    $this->em->flush();

        return $this->redirectToRoute('cart_success', [
            'orderNumber' => $purchase->getOrderNumber(),
        ]);
    }
```

j. Extraits de code des controllers montrant les routes user et admin

Route où l'on voit bien que le ROLE_USER

```php
#[IsGranted('ROLE_USER')]
class LessonController extends AbstractController
{
    public function __construct(…
    ) {
    }

    /**…
    private function getUserCompletedLessonMap(User $user): array
    {…
    }

    #[Route('/lesson/{id}', name: 'lesson_show', methods: ['GET'])]
    public function show(int $id, LessonRepository $lessonRepository): Response
    {
        if ($this->isGranted('ROLE_ADMIN')) {
            return $this->redirectToRoute('admin_dashboard');
        }

        $lesson = $lessonRepository->findVisibleLesson($id);

        if (!$lesson) {
            throw $this->createNotFoundException('Leçon introuvable.');
        }

        $user = $this->getUser();
        if (!$user instanceof User) {
            return $this->redirectToRoute('app_login');
        }

        if (!$this->access->userCanAccessLesson($user, $lesson)) {
            $this->addFlash('danger', "Tu n'as pas accès à cette leçon.");
            return $this->redirectToRoute('cursus_show', [
                'id' => $lesson->getCursus()?->getId(),
            ]);
        }

        $userHasCompleted = $this->getUserCompletedLessonMap($user);

    $certification = $this->em->getRepository(Certification::class)->findOneBy([
        'user' => $user,
        'lesson' => $lesson,
        'type' => 'lesson',
    ]);

    return $this->render('lesson/show.html.twig', [
        'lesson' => $lesson,
        'userHasAccess' => [$lesson->getId() => true],
        'userHasCompleted' => $userHasCompleted,
        'certification' => $certification,
    ]);
}
```

Route où l’on voit par contre que c’est pour ROLE_ADMIN

```
#[IsGranted('ROLE_ADMIN')]
#[Route('/admin/themes', name: 'admin_theme_')]
class AdminThemeController extends AbstractController
{
    #[Route('', name: 'index', methods: ['GET'])]
    public function index(Request $request, ThemeRepository $repo): Response
    {
        $q = $request->query->get('q');
        $status = $request->query->get('status', 'all');
        $sort = $request->query->get('sort', 'created_desc');

        $requireCursus = $status === 'active';

        $themesWithVisibility = $repo->findAdminThemesWithVisibility(
            $q,
            $status,
            $sort,
            true,
            $requireCursus
        );

        return $this->render('admin/theme/index.html.twig', [
            'themesWithVisibility' => $themesWithVisibility,
            'filters' => [
                'q' => $q,
                'status' => $status,
                'sort' => $sort,
            ],
        ]);
    }
```

k. Flux d’achats d’une formation

```
Utilisateur
    |
    ▼
Consultation du catalogue
(Themes → Cursus → Lessons)
    |
    ▼
Ajout d'une leçon ou d'un cursus au panier
    |
    ▼
Création ou mise à jour d'une commande
(Purchase statut = cart)
    |
    ▼
Enregistrement de l'article
(PurchaseItem)
    |
    ▼
Validation du paiement
    |
    ▼
Commande confirmée
(Purchase statut = paid)
    |
    ▼
Accès aux leçons achetées
    |
    ▼
Validation des leçons
    |
    ▼
Obtention d'une certification
```

I. Diagramme des rôles utilisateurs

```
                Utilisateur
                     |
                     ▼
                ROLE_USER
                     |
        ┌────────────┴────────────┐
        |                         |
        ▼                         ▼
   Accès aux cours            ROLE_ADMIN
   - Thèmes                   Administrateur
   - Cursus                   - Gestion utilisateurs
   - Leçons                   - Gestion formations
   - Panier                   - Gestion achats
   - Dashboard                - Gestion messages
        |
        ▼
User Archived
Compte désactivé
Accès bloqué
```

m. Exemple d'entité Doctrine User

```
#[ORM\Entity(repositoryClass: UserRepository::class)]
#[ORM\Table(name: '`user`')]
#[UniqueEntity(fields: ['email'], message: 'Cet email est déjà utilisé.')]
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 180, unique: true)]
    #[Assert\NotBlank(message: 'This value should not be blank.')]
    #[Assert\Email( ...
    )]
    private ?string $email = null;

    /** ...
    #[ORM\Column]
    private array $roles = [];

    #[ORM\Column]
    private ?string $password = null;

    // Non persisté
    private ?string $plainPassword = null;

    #[ORM\Column(length: 255)]
    #[Assert\NotBlank(message: 'This value should not be blank.')]
    #[Assert\Length(min: 2, max: 255)]
    private ?string $firstName = null;

    #[ORM\Column(length: 255)]
    #[Assert\NotBlank(message: 'This value should not be blank.')]
    #[Assert\Length(min: 2, max: 255)]
    private ?string $lastName = null;

    #[ORM\Column(options: ['default' => false])]
    private bool $isVerified = false;

    #[ORM\Column(length: 255, nullable: true)]
    private ?string $verificationToken = null;
```

```php
    private ?\DateTimeInterface $verificationTokenExpiresAt = null;

    // nullable pour migration / legacy DB
    #[ORM\Column(type: 'datetime_immutable', nullable: true)]
    private ?\DateTimeImmutable $createdAt = null;

    #[ORM\Column(type: 'datetime_immutable', nullable: true)]
    private ?\DateTimeImmutable $archivedAt = null;

    // --- Relations ---
    #[ORM\OneToMany(mappedBy: 'user', targetEntity: LessonValidated::class, cascade: ['remove'], orphanRemoval: tr
    private Collection $lessonValidated;

    #[ORM\OneToMany(mappedBy: 'user', targetEntity: Purchase::class, orphanRemoval: true)]
    private Collection $purchases;

    #[ORM\OneToMany(mappedBy: 'user', targetEntity: Certification::class, orphanRemoval: true)]
    private Collection $certifications;

    #[ORM\ManyToMany(targetEntity: Lesson::class)]
    #[ORM\JoinTable(name: 'user_completed_lessons')]
    private Collection $completedLessons;

    public function __construct()
    {
        $this->lessonValidated = new ArrayCollection();
        $this->purchases = new ArrayCollection();
        $this->certifications = new ArrayCollection();
        $this->completedLessons = new ArrayCollection();

        // Valeurs par défaut
        $this->roles = [];
        $this->createdAt = new \DateTimeImmutable();
    }

    // --- ID & Email ---
    public function getId(): ?int
    {…
    }

    public function getEmail(): ?string
    {…
    }
```

```php
public function setEmail(?string $email): self
{
    $this->email = $email;
    return $this;
}

public function getUserIdentifier(): string
{
    return (string) $this->email;
}

// --- Roles ---

/**...
public function getRoles(): array
{
    $roles = $this->roles;
    $roles[] = 'ROLE_USER';

    return array_values(array_unique($roles));
}

/**...
public function setRoles(array $roles): self
{...
}

/**...
public function getStoredRoles(): array
{...
}

/**...
public function setStoredRoles(array $roles): self
{...
}

// --- Password ---
public function getPassword(): ?string
{...
}
```

n. Exemple de configuration de sécurité

Extrait de Security.yaml

```yaml
access_control:
    # Pages publiques
    - { path: ^/login, roles: PUBLIC_ACCESS }
    - { path: ^/register, roles: PUBLIC_ACCESS }
    - { path: ^/logout, roles: PUBLIC_ACCESS }

    # Dashboard réservé aux utilisateurs connectés
    - { path: ^/dashboard, roles: ROLE_USER }

    # Zone admin
    - { path: ^/admin, roles: ROLE_ADMIN }
```

Extrait de l'UserChecker

```php
class UserChecker implements UserCheckerInterface
{
    public function checkPreAuth(UserInterface $user): void
    {
        if (!$user instanceof User) {
            return;
        }

        if ($user->isArchived()) {
            throw new CustomUserMessageAccountStatusException(
                'Votre compte est archivé. Contactez un administrateur.'
            );
        }

        if (!$user->isVerified()) {
            throw new CustomUserMessageAccountStatusException(
                'Votre compte n'est pas encore vérifié.'
            );
        }
    }

    public function checkPostAuth(UserInterface $user): void
    {
        // Rien ici
    }
}
```

o. Exemple de test automatisé PHPUnit

Extrait de LessonControllerTest montrant un scénario de validation de leçon et création de certificat

```php
public function testCompleteMarksLessonCompletedAndCreatesCertification(): void
{
    $user = $this->getFixtureUser();
    $lesson = $this->getFixtureLesson();

    $this->createPaidPurchaseForLesson($user, $lesson);

    $user = $this->em->getRepository(User::class)->find($user->getId());
    $lesson = $this->em->getRepository(Lesson::class)->find($lesson->getId());

    self::assertNotNull($user);
    self::assertNotNull($lesson);

    $this->client->loginUser($user);

    $token = $this->fetchCsrfTokenFromLessonPage($lesson->getId());

    $this->client->request('POST', '/lesson/' . $lesson->getId() . '/complete', [
        '_token' => $token,
    ]);

    self::assertResponseRedirects('/lesson/' . $lesson->getId());

    $this->client->followRedirect();
    self::assertResponseIsSuccessful();
    self::assertSelectorTextContains('body', 'Leçon marquée comme complétée et certification générée !');

    $this->em->clear();

    $userReloaded = $this->em->getRepository(User::class)->find($user->getId());
    $lessonReloaded = $this->em->getRepository(Lesson::class)->find($lesson->getId());

    self::assertNotNull($userReloaded);
    self::assertNotNull($lessonReloaded);
```

```php
    $cert = $this->em->getRepository(Certification::class)->findOneBy([
        'user' => $userReloaded,
        'lesson' => $lessonReloaded,
        'type' => 'lesson',
    ]);

    self::assertNotNull($cert);
    self::assertNotEmpty($cert->getCertificateCode());
}
```

p. Exemple de data fixtures utilisée pour les tests (ThemeFixtures)

```php
class ThemeFixtures extends Fixture
{
    public const THEME_MUSIQUE_REF = 'theme_musique';
    public const CURSUS_GUITARE_REF = 'cursus_guitare';

    public const LESSON_GUITAR_1_REF = 'lesson_guitar_1';
    public const LESSON_GUITAR_2_REF = 'lesson_guitar_2';

    public function load(ObjectManager $manager): void
    {
        $faker = Factory::create('fr_FR');

        // === Thème Musique ===
        $theme = new Theme();
        $theme->setName('Musique')
            ->setDescription('Apprenez la guitare et le piano')
            ->setImage('images/themes/musique/cours_musique.jpg');
        $manager->persist($theme);
        $this->addReference(self::THEME_MUSIQUE_REF, $theme);

        // Cursus guitare
        $cursusGuitare = new Cursus();
        $cursusGuitare->setName('Cursus d'initiation à la guitare')
            ->setPrice(50)
            ->setTheme($theme)
            ->setImage('images/themes/musique/cours_guitare.jpg');
        $manager->persist($cursusGuitare);
        $this->addReference(self::CURSUS_GUITARE_REF, $cursusGuitare);

        // Leçons guitare
        $lesson1 = new Lesson();
        $lesson1->setTitle('Découverte de l'instrument')
            ->setPrice(26)
            ->setCursus($cursusGuitare)
            ->setImage('images/themes/musique/cours_guitare.jpg')
            ->setFiche($this->generateArticles($faker))
            ->setVideoUrl('https://youtu.be/example1');
        $manager->persist($lesson1);
        $this->addReference(self::LESSON_GUITAR_1_REF, $lesson1);
```

q. Schéma expliquant l'organisation des tests effectués sur l'application

```
tests/
├── Admin/        → tests du back-office
├── Controller/   → tests des contrôleurs front-end
├── Command/      → tests des commandes CLI
├── Entity/       → tests de la logique métier des entités
├── Form/         → tests de validation des formulaires
├── Repository/   → tests des requêtes Doctrine
├── Service/      → tests de la logique applicative
├── Security/     → tests de la sécurité (authentification, voters, etc.)
├── Twig/         → tests du rendu des templates
└── Workflow/     → tests des workflows métier
```